

TALENT INTELLIGENCE PLATFORM

Complete Backend Engineering Interview Handbook

A comprehensive project-based interview guide designed to prepare you for Java Backend Developer, Software Engineer, System Design, and Microservices rounds.

This handbook maps every technical and architectural detail of your self-hosted platform: Java 21, Spring Boot 3.4.0, Spring Cloud, Spring AI, PostgreSQL, PgVector, Redis, Kafka, and Ollama.

CONFIDENTIAL SYSTEM ARCHITECTURE HANDBOOK
DESIGNED FOR HIRING PREPARATION & SYSTEM AUDIT

SECTION 1 - PROJECT INTRODUCTION

Question: Tell me about your project.

* Spoken Answer:

I built a web application called the Talent Intelligence Platform. It helps recruiters automate candidate hiring. When a recruiter uploads candidate resumes, the system parses them, extracts details like skills and experience, and ranks candidates against job descriptions. It also has an AI copilot chat where recruiters can query candidate profiles and generate interview questions using natural language.

* Technical Answer:

I designed and implemented the Talent Intelligence Platform, a distributed, event-driven microservices architecture built on Java 21 and Spring Boot 3.4. The platform automates candidate resume ingestion, semantic parsing, candidate-to-job match scoring, and real-time copilot interactions using Retrieval-Augmented Generation (RAG). Infrastructure includes Spring Cloud Gateway, Eureka Server, Apache Kafka, PostgreSQL with PgVector, Redis, and local LLM execution via Ollama.

* Real Interview Answer:

In my project, we address the bottleneck of manual resume screening. When a resume is uploaded, it is stored in AWS S3, and a Kafka event triggers an asynchronous pipeline. The AI Screening service uses Apache Tika to parse the file, splits it into semantic chunks, and generates 768-dimensional embeddings using `nomic-embed-text` which are saved in a PgVector database. A candidate ranking service computes match scores against job requirements using Ollama Llama3, caching ranks in a Redis Sorted Set for sub-millisecond retrieval. Finally, we provide a RAG-enabled Recruiter Chat Service with query rewriting, expansion, hybrid retrieval, and cosine reranking, backed by Redis-managed chat memory.

* Follow-Up Questions:

- How do you handle failure if the local LLM is down?
- What chunk size and overlap did you choose for text processing, and why?

* Common Mistakes:

Failing to mention Java 21 virtual threads or forgetting to list PgVector and Kafka as key infrastructure decoupling points.

* Interview Tips:

Start with the high-level business problem (recruiter fatigue), then immediately transition into the engineering solutions (event-driven parsing, vector search, Redis caching).

* Cross Questions / Trap Questions:

- Why did you choose a microservices architecture instead of a monolith for this platform? Isn't it overkill for a recruiter dashboard?
- If two recruiters upload resumes simultaneously, how does your system scale?

* Best Responses:

- Microservices were chosen to decouple the high-CPU parsing and LLM inference services from the transactional job and user management services, ensuring that resume ingestion spikes don't bring down user login.
- Concurrency is managed by Kafka partitions and Spring Boot virtual threads, allowing horizontal scaling of the AI Screening consumer group.

Question: Explain your project in 30 seconds.*** Spoken Answer:**

It is an AI-powered recruiter dashboard where recruiters upload resumes and instantly get candidate rankings against job profiles. It also includes an AI assistant that can answer questions about the candidates' resumes and write customized interview questions.

*** Technical Answer:**

It is a distributed microservices platform built on Java 21, Spring Boot, and Spring Cloud. It features an event-driven ingestion pipeline that parses resumes via Apache Tika, indexes them in a PostgreSQL PgVector database, ranks them against vacancies in a Redis Sorted Set cache, and provides an interactive RAG copilot chatbot using Spring AI and Ollama Llama3.

*** Real Interview Answer:**

My project is a cloud-native Talent Intelligence Platform. It implements a decoupled microservices architecture to process resumes. The core engine runs Apache Tika for text extraction, chunking, and PgVector for embedding storage. Candidate ranks are computed via local LLM and cached in Redis ZSet. The interface exposes a RAG-based recruiter copilot with query expansion and hybrid search.

*** Follow-Up Questions:**

- What is the throughput of your resume processing pipeline?
- Which database stores the primary candidate profiles?

*** Common Mistakes:**

Getting bogged down in UI details; focus on the backend microservices.

*** Interview Tips:**

Structure: 1. Core Stack -> 2. Business Value -> 3. High-performance Component.

*** Cross Questions / Trap Questions:**

- How does the API Gateway authenticate these requests in under 30 seconds?

*** Best Responses:**

- The API Gateway performs local JWT validation, propagating claims via headers, which avoids database round-trips and keeps latency under 5 milliseconds.

SECTION 2 - COMPLETE ARCHITECTURE EXPLANATION

Question: Describe the End-to-End Request Flow of a Resume Upload.

* Spoken Answer:

The user uploads a file from the frontend. The request passes through the API Gateway, which checks if the user is logged in. It then routes the file to the Resume Service, which uploads it to S3, saves a database record as 'UPLOADED', and sends a Kafka message. The AI Screening service reads the message, parses the text, saves chunks and vector embeddings, and marks it 'PARSED'. Finally, the Ranking service scores it and caches it in Redis, changing the status to 'SCREENED'.

* Technical Answer:

1. Client triggers a POST to `/api/v1/resumes/upload` routing through the Netty-based api-gateway.
2. The gateway's `AuthenticationFilter` intercepts the request, verifies the JWT, and mutates HTTP headers to inject user identity claims.
3. The request is load-balanced via Eureka and forwarded to `resume-management-service`.
4. The service uploads the file binary to S3 using the AWS SDK, records a `CandidateResume` entity with status `UPLOADED`, and publishes a `resume-uploaded` event containing the `resumeld` and `fileUrl` to Kafka.
5. `ai-screening-service` consumes this event, downloads the S3 file, extracts text via Tika, chunks it via `TextChunker`, generates vector embeddings, stores them in `screening_db` and `PgVector`, publishes a `resume-parsed` event, and calls `resume-management-service` via Feign to update status to `PARSED`.
6. `candidate-ranking-service` consumes `resume-parsed`, fetches the job description, scores candidate fit via Ollama Llama3, saves the `ScreeningReport`, adds the score to Redis ZSet (`jd:ranking:{jobId}`), and publishes a status update to mark the resume `SCREENED`.

* Real Interview Answer:

In our architecture, the upload path transitions from a synchronous API call to an asynchronous pipeline. By uploading the raw file to S3 and returning a 202 Accepted, we keep the UI responsive. The heavy work (Tika parsing, semantic chunking, vector embedding, and LLM matchmaking evaluation) is deferred to Kafka event-driven consumers, ensuring system stability during high loads.

* Follow-Up Questions:

- What happens if the gateway-injected headers are spoofed by an external client?
- How do you prevent Kafka message duplication in the AI Screening consumer?

* Common Mistakes:

Forgetting to mention the database-per-service pattern or failing to explain the Gateway's claims injection.

* Interview Tips:

Sketch the architecture: Gateway -> Service A -> Kafka -> Service B, labeling each connection as Sync (REST/Feign) or Async (Kafka).

* Cross Questions / Trap Questions:

- If the Feign call to update status to `PARSED` fails, how does the system recover?

* Best Responses:

- The gateway is the only service exposed to the internet. Downstream services run behind a private virtual network and reject any external requests. Additionally, our `GatewayHeaderAuthFilter` will only accept headers from trusted gateway IPs.
- If the Feign client fails, the status update method catches the exception and publishes a `resume-status-updated` event to Kafka. The `resume-management-service` consumes this topic asynchronously, guaranteeing eventual

consistency.

SECTION 3 - MICROSERVICES INTERVIEW ROUND

Question: Why did you choose the Database-per-Service Pattern and how do you handle joins?

* Spoken Answer:

We give every microservice its own database. For example, User Service has user_db and Resume Service has resume_db. This stops services from locking each other's databases. If we need data from another service, we use REST calls (Feign clients) or listen to Kafka events instead of joining tables directly.

* Technical Answer:

The Database-per-Service pattern was chosen to ensure loose coupling, database schema independence, and service scalability. It prevents tight coupling at the database layer. We handle distributed queries in two ways: 1. Synchronous REST Queries: We use Spring Cloud OpenFeign to retrieve metadata dynamically (e.g., candidate-ranking-service queries job-description-service for job profiles). 2. Asynchronous Event Replication: We replicate lookup data using Kafka events. For instance, when a recruiter verifies registration, user-management-service consumes the USER_REGISTERED event to populate its local database.

* Real Interview Answer:

In our architecture, domain isolation is strictly enforced. For example, recruiter-chat-service requires candidate profile information. Instead of direct database joins (which would bind the Chat database to the Resume database), the Chat service calls the Resume service via OpenFeign. For hybrid search keyword fallback, rather than querying the resume_chunks table directly, it makes a REST call to the ai-screening-service which owns that table.

* Follow-Up Questions:

- How do you handle distributed transactions across these isolated databases?
- What caching strategy do you use to optimize Feign client latency?

* Common Mistakes:

Suggesting a shared database with separate schemas; in a true microservices setup, database engines must be completely isolated.

* Interview Tips:

Emphasize that coupling at the database is the hardest coupling to break. Show how your Kafka events bypass this coupling.

* Cross Questions / Trap Questions:

- If you are doing REST queries for every join, isn't your system bottlenecked by network latency and cascading failures?

* Best Responses:

- We mitigate cascading failures using Resilience4j circuit breakers and fallback responses. To minimize network overhead, frequently accessed read-heavy structures (like candidate ranking lists) are cached in Redis.

SECTION 4 - SPRING BOOT INTERVIEW ROUND

Question: Explain the lifecycle of a Spring Bean and how it applies to your configurations.

*** Spoken Answer:**

Spring beans are Java objects managed by the Spring IoC container. The lifecycle starts with instantiation, followed by dependency injection. Then, post-processors run, init methods are called, and the bean is ready. When the application stops, destroy methods are executed to clean up resources.

*** Technical Answer:**

A Spring bean lifecycle consists of: 1. Instantiation (creating bean instance) 2. Populate Properties (Dependency Injection) 3. Aware Interfaces (calling setters for aware interfaces) 4. BeanPostProcessor Pre-Initialization 5. Custom Initialization (methods annotated with `@PostConstruct` or implementing `InitializingBean`) 6. BeanPostProcessor Post-Initialization (creating AOP proxies) 7. Ready for Use 8. Custom Destruction (`@PreDestroy` or implementing `DisposableBean`).

*** Real Interview Answer:**

In my platform, this lifecycle is critical. For instance, our AdminSeeder uses `@PostConstruct` to populate initial admin roles in the database immediately after bean initialization and dependency injection have completed.

*** Follow-Up Questions:**

- Why is Constructor Injection preferred over Field Injection?
- How does Spring resolve circular dependencies?

*** Common Mistakes:**

Confusing bean lifecycle with the JVM classloader lifecycle.

*** Interview Tips:**

Mention `@PostConstruct` and `@PreDestroy` as standard annotations you use daily.

*** Cross Questions / Trap Questions:**

- What happens if a prototype-scoped bean is injected into a singleton-scoped bean? How does the lifecycle change?

*** Best Responses:**

- The prototype bean is instantiated only once during the initialization of the singleton bean. To get new instances of the prototype bean dynamically, we must use method injection or a Provider or ObjectFactory.

SECTION 5 - POSTGRESQL INTERVIEW ROUND

Question: What are database Isolation Levels, and how do they prevent anomalies?

* Spoken Answer:

Isolation levels define how visible changes made by one database transaction are to other running transactions. PostgreSQL has four levels: Read Uncommitted, Read Committed, Repeatable Read, and Serializable. They prevent issues like reading dirty data, non-repeatable reads, and phantom reads.

* Technical Answer:

SQL isolation levels control concurrency anomalies. PostgreSQL defaults to Read Committed: Dirty Reads are prevented natively. Non-Repeatable Reads are prevented in Repeatable Read isolation level using Multi-Version Concurrency Control (MVCC) snapshots. Phantom Reads are also prevented in PostgreSQL's Repeatable Read implementation. Serialization Anomalies are prevented in Serializable level using SSI (Serializable Snapshot Isolation) locks.

* Real Interview Answer:

In our candidate screening system, when calculations are done in parallel, we use repeatable read isolation level to ensure that reads of job parameters remain consistent throughout the screening calculation, avoiding anomalies if a recruiter edits the job details simultaneously.

* Follow-Up Questions:

- How does Multi-Version Concurrency Control (MVCC) work in PostgreSQL?
- What is the performance overhead of Serializable isolation level?

* Common Mistakes:

Believing that higher isolation levels run faster; higher levels increase lock contention.

* Interview Tips:

Memorize the anomalies prevented by each level: Dirty Reads, Non-repeatable Reads, Phantom Reads.

* Cross Questions / Trap Questions:

- Why does PostgreSQL's Repeatable Read level prevent phantom reads, unlike standard SQL specifications?

* Best Responses:

- PostgreSQL implements Repeatable Read using MVCC snapshots. A transaction sees data from a snapshot taken when the transaction began, which inherently blocks new inserts from appearing in ongoing queries.

SECTION 6 - REDIS INTERVIEW ROUND

Question: How does the Candidate Ranking system use Redis Sorted Sets (ZSet)?

* Spoken Answer:

Redis Sorted Sets are data structures that hold unique members, where each member is mapped to a numeric score. In our ranking system, the members are candidate resume IDs, and the scores are the candidate match scores. We key this set by the job ID. This lets us query candidate lists sorted by score instantly.

* Technical Answer:

In candidate-ranking-service, we cache active candidate scores inside Redis Sorted Sets (ZSets) using the key schema `jd:ranking:{jobId}`. When a candidate's match score is calculated, we execute a `ZADD` operation: `redisTemplate.opsForZSet().add('jd:ranking:' + jobId, resumeId, matchScore)`. To retrieve the top candidates sorted by score in descending order, we run a `ZREVRANGEBYSCORE` or `ZREVRANGE` command, providing the offset and limit for pagination. This delivers $O(\log(N) + M)$ complexity.

* Real Interview Answer:

We implement a write-through caching strategy. The primary report is written to PostgreSQL for durability, while the ZSet in Redis acts as the high-speed index for the recruiter frontend dashboard. This allows recruiters to view candidate lists sorted by matching compatibility in sub-milliseconds under high user concurrency.

* Follow-Up Questions:

- How do you invalidate this cache if a resume is deleted?
- What happens if Redis goes down? How does the ranking service recover?

* Common Mistakes:

Storing the entire candidate profile JSON in the ZSet; only the ID and score should be stored to optimize memory.

* Interview Tips:

Use commands in your explanation (e.g. `ZADD`, `ZREVRANGE`) to prove hands-on Redis experience.

* Cross Questions / Trap Questions:

- If two candidates have the same score, how does Redis order them? How do you guarantee deterministic ordering?

* Best Responses:

- Lexicographical sorting is applied by Redis to tie-breaker members with identical scores. To enforce a custom deterministic sort, we fall back to a database query sorted by score and creation timestamp.

SECTION 7 - KAFKA INTERVIEW ROUND

Question: How do you implement Idempotent Consumers in your microservices?

* Spoken Answer:

An idempotent consumer ensures that even if a message is received more than once, it is only processed once. We do this by checking if the message has already been processed before running any business logic. For example, in our User Service, we check if the recruiter profile already exists in the database. If it does, we just ignore the message.

* Technical Answer:

Due to Kafka's at-least-once delivery guarantee, network retries can cause duplicate messages. We enforce idempotency using the Unique Identifier / Database Constraint Pattern: 1. Each event contains a business-key identifier (e.g., `resumeld` or `email`). 2. On consuming an event, the service executes a read check (e.g., `recruiterRepository.existsByEmail(email)`). 3. The database schema enforces a unique constraint on these columns. 4. If a duplicate event bypasses the application check due to race conditions, the database throws a `UniqueConstraintViolationException`, which is caught and acknowledged safely without duplicate processing.

* Real Interview Answer:

Our consumer endpoints are built to be idempotent. In `AuthEventConsumer.java`, we check `recruiterRepository.existsByEmail(email)` before profile creation. If true, we log it and return. This protects us from duplicate profile creation errors.

* Follow-Up Questions:

- What is the performance overhead of doing a read-before-write check on every consumer message?
- How does Kafka's idempotent producer config differ from consumer-side idempotency?

* Common Mistakes:

Relying only on application-level checks without database-level unique constraints.

* Interview Tips:

Always mention 'read-before-write' and 'database unique constraints' as the double-layer defense for idempotency.

* Cross Questions / Trap Questions:

- What happens if two consumers in the same group read duplicate messages simultaneously before the database transaction commits?

* Best Responses:

- We prevent this race condition by using the consumer message key (e.g., `email` or `resumeld`) as the Kafka partition key. This guarantees that all messages related to a specific entity are routed to the same partition and processed sequentially by a single consumer thread.

SECTION 8 - AI SCREENING SERVICE ROUND

Question: How does the AI Screening Service extract structured metadata from raw resume text?

*** Spoken Answer:**

We use Spring AI to call a local LLM model (Llama3) hosted on Ollama. We send the extracted resume text along with a strict prompt that tells the model to extract fields like name, email, experience, and skills, and return the data only as a valid JSON object. We then parse this JSON in Java using Jackson.

*** Technical Answer:**

The ai-screening-service uses Spring AI's ChatClient to interface with Ollama. To extract metadata, we pull a 15,000-character excerpt of the parsed text. The prompt mandates a JSON schema containing name, email, totalExperience, skills, and noticePeriod. We configure fallback mechanisms: if the LLM output contains markdown ticks or conversational text, we sanitize it using regex string parsing. Finally, Jackson maps the JSON to our ResumeMetadataDto. If the model fails or is offline, we run regex checks for the candidate's email and set placeholders.

*** Real Interview Answer:**

In my code, we do regex cleanup on the raw model output to handle JSON schema failures. If LLM details extraction fails entirely, we run standard regex check to extract email addresses and fill in placeholders, ensuring the ingestion pipeline doesn't block.

*** Follow-Up Questions:**

- Why do you use Llama3 instead of a smaller, faster model like Phi-3 for metadata extraction?
- How does the system handle resumes written in multiple columns or complex layouts?

*** Common Mistakes:**

Expect the LLM to output 100% clean JSON without parsing fallback code in Java.

*** Interview Tips:**

Highlight the regex fallback code for extracting email addresses; it shows production resilience.

*** Cross Questions / Trap Questions:**

- What happens if the candidate writes fake information in the resume? How does your AI detect it?

*** Best Responses:**

- The AI screening service evaluates internal consistency (comparing project timelines against total experience claims) and lists gaps. Verification of factual details is left to background verification checks during later stages.

SECTION 9 - RAG INTERVIEW ROUND

Question: Describe your RAG Pipeline's Retrieval and Reranking architecture.

* Spoken Answer:

When a recruiter asks a question about a candidate, we don't send the entire resume to the AI because it's too large. Instead: 1. We expand the recruiter's question into multiple variations. 2. We retrieve the most relevant text chunks from PgVector. 3. We merge and deduplicate the chunks using RRF. 4. We rerank these chunks using cosine similarity to select the top 5. 5. We inject these 5 chunks into the prompt as context and ask the LLM to answer the question based only on this context.

* Technical Answer:

Our RAG pipeline is designed to maximize retrieval recall and precision: 1. Query Rewriting: The original query is optimized by the LLM for semantic search. 2. Multi-Query Expansion: We generate 3 query variants. 3. Hybrid Retrieval: We execute PgVector similarity searches. If vector matching returns less than 2 chunks, we execute a database keyword search fallback. 4. Reciprocal Rank Fusion (RRF): Merges lists from all query variants, boosting documents that appear frequently. 5. Cosine Reranker: Computes cosine similarity between the query embedding and retrieved chunk embeddings, sorting and selecting the top 5 context chunks. 6. Context Injection: The selected text is injected into the LLM system prompt along with history retrieved via RedisChatMemory.

* Real Interview Answer:

In recruiter-chat-service, we execute a highly sophisticated RAG pipeline: RagPipelineService.java coordinates query rewriting, expansion, hybrid retrieval, RRF merging, and Cosine reranking to feed Llama3 the top 5 most relevant chunks.

* Follow-Up Questions:

- What is the latency overhead of executing query expansion and reranking?
- Why is Reciprocal Rank Fusion useful for multi-query retrieval?

* Common Mistakes:

Confusing query expansion with query rewriting; expansion creates multiple queries, while rewriting refines a single query.

* Interview Tips:

Walk through the pipeline step-by-step: User Query -> Query Expansion -> Retrieval -> RRF -> Reranking -> LLM Generation.

* Cross Questions / Trap Questions:

- If the retrieved context is too large, how do you prevent context truncation errors in the LLM?

* Best Responses:

- We limit retrieval using RRF and a cosine reranker, selecting only the top 5 most relevant chunks (approx. 4,000 characters). This fits easily within Llama3's context window.

SECTION 10 - SYSTEM DESIGN ROUND

Question: How does your system design handle the CAP Theorem trade-offs?

* Spoken Answer:

CAP states a system can only guarantee two out of Consistency, Availability, and Partition Tolerance. In our distributed microservices, we prioritize Availability and Partition Tolerance (AP). This means we use asynchronous Kafka events to process data in the background, allowing the system to keep running even if some services are temporarily unavailable.

* Technical Answer:

In a distributed microservices architecture, network partitions are inevitable, meaning we must choose between Consistency (C) and Availability (A). We choose an AP (Eventual Consistency) design: Availability: Core user features (uploading resumes, browsing jobs) remain active. Partition Tolerance: Services operate independently using their own databases. Eventual Consistency: Kafka processes asynchronous events. The state of candidate match scores and profiles syncs across services eventually.

* Real Interview Answer:

In our design, eventual consistency is key. If candidate-ranking-service goes down, recruiters can still upload resumes. When the ranking service restarts, it consumes the backlog of resume-parsed events from Kafka, catching up to database consistency without downtime.

* Follow-Up Questions:

- How do you handle write conflicts in an eventually consistent model?
- What reconciliation workers do you run to verify consistency?

* Common Mistakes:

Claiming your entire microservice system is 100% consistent (ACID); distributed systems rely on eventual consistency.

* Interview Tips:

State clearly: 'We choose AP for scale and ingest pipelines, and CP for authentication and authorization.'

* Cross Questions / Trap Questions:

- If a recruiter deletes a resume, but the database deletion event is lost in partition, how do you prevent the candidate from appearing in rankings?

* Best Responses:

- We enforce soft deletes at the domain level and run a daily cron job that audits S3 references against PostgreSQL and Redis indexes, removing any orphan records.

SECTION 11 - KUBERNETES ROUND

Question: What are Liveness and Readiness Probes, and how are they configured in your services?

*** Spoken Answer:**

Probes are health checks run by Kubernetes. A Liveness Probe checks if a service is alive. If it fails, Kubernetes restarts the pod. A Readiness Probe checks if a service is ready to take user traffic. If it fails, Kubernetes stops sending requests to that pod. We configure these using Spring Boot Actuator endpoints.

*** Technical Answer:**

We configure liveness and readiness probes inside our deployments using Spring Boot Actuator: Liveness Probe: Queries `/actuator/health/liveness`. It verifies the application context is active. Readiness Probe: Queries `/actuator/health/readiness`. It checks resource connections (database pool, Redis connectivity, Kafka connection status).

*** Real Interview Answer:**

In our `services-deployments.yaml` file, we configure the probes with an `initialDelaySeconds` of 30-40 seconds. This is critical to prevent Kubernetes from killing Java pods during Spring Boot context cold startup.

*** Follow-Up Questions:**

- What happens if a database connection times out temporarily? Will the readiness probe cause the pod to restart?
- How do you tune probe parameters like `failureThreshold` and `timeoutSeconds`?

*** Common Mistakes:**

Setting the liveness probe to check external database dependencies; liveness should only check the local pod context.

*** Interview Tips:**

Differentiate clearly: Liveness = Restart Pod, Readiness = Remove from Load Balancer.

*** Cross Questions / Trap Questions:**

- If all pods of a service fail the readiness probe because the database is down, how does Kubernetes handle client traffic?

*** Best Responses:**

- Kubernetes removes all pods from the Service endpoints list. Traffic hitting the Ingress receives a 503 Service Unavailable, preventing requests from failing silently inside broken pods.

SECTION 12 - OBSERVABILITY ROUND

Question: Describe your Monitoring and Observability stack.

* Spoken Answer:

We monitor our system using Prometheus and Grafana. Spring Boot Actuator exposes metrics about memory, CPU, and database pools. Prometheus scrapes these metrics at regular intervals. Grafana pulls data from Prometheus and displays it on visual dashboards, helping us see JVM health, Kafka queue lag, and DB query times.

* Technical Answer:

Our observability architecture is built on a pull-based metrics collection model: 1. Instrumentation: Services run Micrometer and Spring Boot Actuator, exposing Prometheus metrics at /actuator/prometheus. 2. Aggregation: Prometheus collects metrics from all microservices, exporters (Postgres, Redis, Kafka exporters), and cAdvisor container stats. 3. Visualization: Grafana provisions dashboards for JVM health, Kafka consumer lag, HikariCP connection pool usage, and custom business metrics (e.g., ranking.redis.updates.total). 4. Alerting: Alertmanager triggers notifications.

* Real Interview Answer:

Our monitoring covers both infrastructure and application layers. For example, candidate-ranking-service increments Prometheus counters on Redis updates. Grafana monitors these metrics to detect memory bottlenecks.

* Follow-Up Questions:

- How do you measure API response latencies using Prometheus?
- What are the key metrics to monitor in a Kafka consumer?

* Common Mistakes:

Confusing logging (Logback) with metrics (Prometheus).

* Interview Tips:

Mention specific metrics you monitor: JVM Heap memory, Kafka consumer lag, and database connection pool saturation.

* Cross Questions / Trap Questions:

- If Prometheus scraping adds CPU overhead to your services, how do you optimize it?

* Best Responses:

- We optimize scraping by extending the scrape interval to 15 or 30 seconds and disabling non-essential Actuator metrics (like detailed file system stats) in the properties configuration.

SECTION 13 - PRODUCTION READINESS ROUND

Question: How do you handle Circuit Breakers and Retry patterns in Service-to-Service communication?

* Spoken Answer:

A Circuit Breaker prevents a service from calling another service if it knows the target service is down. This stops threads from getting stuck. We use Resilience4j with OpenFeign. If a call fails, we execute a fallback method that returns cached or empty data instead of throwing an error.

* Technical Answer:

In distributed systems, cascading failures occur when a slow service exhausts threads in upstream callers. We mitigate this using Resilience4j: Circuit Breaker: Monitors calls. If the failure rate exceeds 50% within a sliding window of 10 calls, the circuit opens, failing fast without calling the target service. Retry Pattern: For transient network issues, we retry 3 times with exponential backoff before opening the circuit. Fallback: We define fallback methods.

* Real Interview Answer:

In candidate-ranking-service, if the Feign client call to ai-screening-service fails, the Resilience4j fallback method triggers a local cosine similarity search calculation in Java, ensuring candidate ranking remains operational.

* Follow-Up Questions:

- What is the difference between open, closed, and half-open circuit breaker states?
- How do you configure sliding windows (count-based vs time-based) in Resilience4j?

* Common Mistakes:

Setting short timeouts without configuring retries, or writing fallback methods that throw exceptions.

* Interview Tips:

Explain the state transitions of a circuit breaker: Closed -> Open -> Half-Open -> Closed.

* Cross Questions / Trap Questions:

- If your retry pattern triggers on every timeout, won't it overwhelm an already struggling downstream service (retry storm)?

* Best Responses:

- We prevent retry storms by implementing exponential backoff with random jitter, ensuring that retries from multiple pods do not hit the downstream service simultaneously.

SECTION 14 - RECRUITER & HR ROUND

Question: Why did you build this project, and what was your biggest technical challenge?

* Spoken Answer:

I built this project to learn how to design a real-world, large-scale backend system. The biggest challenge was connecting the AI service with the Java backend. Local LLM models are slow, so I had to build a fallback search system using database vectors so the app wouldn't freeze if the AI went offline.

* Technical Answer:

I built this platform to explore the intersection of enterprise Java microservices and local artificial intelligence. My biggest challenge was managing the latency and availability of the local LLM. When multiple resumes were uploaded, synchronous API calls to Ollama caused timeouts. I resolved this by redesigning the pipeline to be event-driven via Kafka, moving the parsing and embedding tasks to asynchronous workers, and implementing a mathematical cosine-similarity search fallback in Java to keep the system operational if the LLM server failed.

* Real Interview Answer:

The key lesson was learning how to design for failure. AI inference is highly CPU-bound and unpredictable. Designing asynchronous processing via Kafka and combining it with a mathematical fallback algorithm taught me how to construct robust, enterprise-grade architectures.

* Follow-Up Questions:

- What did you learn about Java 21 virtual threads during this project?
- How would you scale this platform if the user base grew by 10x?

* Common Mistakes:

Talking about non-technical issues; keep the focus on architectural and system bottlenecks you resolved.

* Interview Tips:

Use the STAR method: Situation (latency), Task (optimize), Action (Kafka + Fallback), Result (improved uptime & throughput).

* Cross Questions / Trap Questions:

- If you had to start this project again, what architecture decision would you change?

* Best Responses:

- I would implement a hybrid storage model from day one, storing small text fragments in Redis cache and streaming large resume files directly to S3 via pre-signed URLs, bypassing the backend server for raw file uploads to save network bandwidth.

SECTION 15 - INTERVIEWER CROSS QUESTIONS

Question: Why did you use Kafka for status updates when you also use Feign clients? Isn't double-writing status updates a design error?

*** Spoken Answer:**

We don't write the status twice at the same time. The Feign client is our primary way to update the status because it is direct and synchronous. We only publish a Kafka event if the Feign call fails. This is a fallback to make sure the status eventually gets updated even if the network is unstable.

*** Technical Answer:**

This is not a double-write; it is a fail-fast REST with asynchronous event fallback pattern. We execute status updates to resume-management-service synchronously via OpenFeign because the UI expects immediate status updates. If the REST call succeeds, the method exits. If the Feign call fails, the exception is caught, and we publish a status update event to the Kafka resume-status-updated topic. The Resume Service consumes this topic asynchronously. This guarantees consistency without blocking threads.

*** Real Interview Answer:**

In AIScreeningService.java, the status update catches Feign client exceptions and publishes to Kafka. This guarantees eventual consistency for the candidate's screening state.

*** Follow-Up Questions:**

- How do you prevent out-of-order status updates in the Kafka consumer?
- What is the timeout configuration for your Feign client?

*** Common Mistakes:**

Admitting it's a design error; explain that it's a deliberate resilience pattern.

*** Interview Tips:**

Always defend your design choices with clear recovery scenarios.

*** Cross Questions / Trap Questions:**

- If the REST call fails, but the Kafka broker is also down, how does the system recover?

*** Best Responses:**

- In that scenario, the resume remains in its last known state. We run a daily reconciliation job that queries pending/uncompleted resume states in PostgreSQL and re-triggers the processing pipeline.

SECTION 17 - WHY QUESTIONS

Question: Why Microservices instead of Monolith for this platform?

*** Spoken Answer:**

Microservices let us split the app into smaller, independent parts. For example, processing resumes with AI takes a lot of CPU power. If we used a monolith, a spike in resume uploads could freeze the login page for all recruiters. By separating them, authentication stays fast and running even during heavy uploads.

*** Technical Answer:**

We chose a microservices architecture to enforce domain boundaries, scale CPU-heavy AI workloads independently from transactional CRUD services, and prevent single points of failure. The AI Screening service relies on Apache Tika and Ollama, which require substantial CPU and RAM. Placing it in its own container allows us to allocate hardware resources specifically to it without impacting other services.

*** Alternative Approaches:**

A monolithic architecture with asynchronous thread pools. However, this locks scaling capabilities and binds the database schemas together, increasing deployment coupling.

*** Trade-offs:**

Microservices introduce network latency, transactional complexity, and management overhead (Eureka, Gateway, Configs). We traded transactional simplicity for system resilience and independent scalability.

*** Cross Questions / Trap Questions:**

- Doesn't the latency of inter-service Feign calls cancel out the performance benefits of microservices?

*** Best Responses:**

- No, because we design heavy processing to be asynchronous via Kafka, leaving synchronous Feign calls only for quick, metadata lookups which are cached in Redis.

Question: Why Kafka instead of RabbitMQ?

*** Spoken Answer:**

We chose Kafka because it is built for high-throughput stream processing and records events permanently. This lets us replay events if a service crashes or if we need to rebuild our databases. RabbitMQ deletes messages after they are read, which doesn't fit our event-sourcing design.

*** Technical Answer:**

Apache Kafka was chosen for its partitioned commit-log architecture, high-throughput capability, and event replay support. Kafka stores events on disk, enabling services to re-read events in case of catastrophic database failure. RabbitMQ, being a traditional message broker, deletes messages immediately upon acknowledgment, which limits retry flexibility and event-driven logging.

*** Alternative Approaches:**

RabbitMQ or AWS SQS. These are easier to configure but lack partition replay log capabilities and stream processing scalability.

*** Trade-offs:**

Kafka has a higher operational complexity (Zookeeper/Kraft coordination) and is heavier on resources compared to RabbitMQ, which we accepted in exchange for event durability and replay support.

*** Cross Questions / Trap Questions:**

- If you only have 10 microservices, do you really have the high throughput that warrants Kafka?

*** Best Responses:**

- While our current throughput is manageable, our platform handles parsing and vectorizing large document batches. Kafka's commit log allows us to scale consumer groups horizontally and replay historical resume data if we change our embedding model in the future.

Question: Why PGVector instead of a dedicated Vector DB like Pinecone?

*** Spoken Answer:**

We chose PGVector because it runs inside PostgreSQL. This means we can keep our normal database tables and our AI embeddings in the same database. If we used a separate vector database like Pinecone, we would have to keep them in sync, which is hard to do and can cause errors.

*** Technical Answer:**

We selected PGVector to maintain a single source of truth and reduce architectural complexity. Storing vectors inside PostgreSQL allows us to run standard relational queries joined with semantic distance checks in a single SQL statement. Dedicated vector databases like Pinecone or Milvus introduce data synchronization latency and lack transactional ACID guarantees across relational and vector states.

*** Alternative Approaches:**

Pinecone, Qdrant, or Milvus. These dedicated stores perform vector index operations faster at extreme scale but add cost and synchronization complexity.

*** Trade-offs:**

PGVector's HNSW index creation can be slower on massive datasets compared to specialized databases, but for candidate screening (scoped per recruiter/job), the relational join capability and transaction consistency outweigh the raw speed difference.

*** Cross Questions / Trap Questions:**

- How do you handle PgVector memory allocation constraints when running inside standard PostgreSQL?

*** Best Responses:**

- We allocate dedicated RAM to PostgreSQL shared buffers and configure pgvector HNSW index parameters specifically, tuning the index size to fit within memory to prevent disk swapping during searches.